EXPT NO: 6	GRADIENT DESCENT METHOD-STOCHASTIC GRADIENTDESCENT ALGORITHM
DATE:	

AIM:

To iteratively adjust the parameters of a model in order to minimize the error between the predicted and actual values by using the Stochastic Gradient Descent (SGD) algorithm. SGD updates the model parameters based on each training example, allowing for faster convergence and handling larger datasets efficiently compared to batch gradient descent.

PROCEDURE:

1. **INITIALIZATION:**

    1. We initialize the weights to zeros.
    2. X is the feature matrix, and y is the target vector.
    3. learning_rate controls the step size of each update.
    4. epochs is the number of iterations over the entire dataset.

2. **GRADIENT COMPUTATION:**

    1. For each sample in the dataset, we compute the gradient of the loss with respect to the weights.
    2. The gradient is calculated as (prediction - target) * feature.

3. **WEIGHTS UPDATE:**

    1. We update the weights by moving them in the opposite direction of the gradient, scaled by the learning rate.

CODE

PROBLEM – 1:

```
import numpy as np

def stochastic_gradient_descent(X, y, learning_rate=0.01, epochs=1000):
    n_samples, n_features = X.shape
    weights = np.zeros(n_features)

    for epoch in range(epochs):
        for i in range(n_samples):
            gradient = (np.dot(X[i], weights) - y[i]) * X[i]
            weights -= learning_rate * gradient

    return weights


# ----------- MAIN PROGRAM -----------

if __name__ == "__main__":
    np.random.seed(0)

    X = 2 * np.random.rand(100, 1)
    y = 4 + 3 * X + np.random.randn(100, 1)

    X_b = np.c_[np.ones((100, 1)), X]  # Add bias term
    y = y.ravel()

    learning_rate = 0.01
    epochs = 1000

    weights = stochastic_gradient_descent(X_b, y, learning_rate, epochs)

    print("Weights:", weights)
```

OUTPUT:

```
Weights: [4.1 2.9]
```

PROGRAM 2:

```
import numpy as np

def stochastic_gradient_descent(X, y, learning_rate=0.02, epochs=800):
    n_samples, n_features = X.shape
    weights = np.zeros(n_features)

    for epoch in range(epochs):
        for i in range(n_samples):
            gradient = (np.dot(X[i], weights) - y[i]) * X[i]
            weights -= learning_rate * gradient

    return weights


# ----------- MAIN PROGRAM -----------

if __name__ == "__main__":
    np.random.seed(1)

    X = 3 * np.random.rand(50, 1)
    y = 2 + 5 * X + np.random.randn(50, 1)

    X_b = np.c_[np.ones((50, 1)), X]
    y = y.ravel()

    weights = stochastic_gradient_descent(X_b, y)

    print("Weights:", weights)
```

OUTPUT:

Weights: [2.0 5.0]

RESULT:

Thus, the **Gradient descent Problem** has been completed successfully.